Name:

Roll no:

Section:

Compiler Construction

Assignment 2

1) Language Overview

Language Name

C++ Bankai Mode

Purpose of the Language

C++ Bankai Mode is a mini C++ like programming language designed for educational purposes - The purpose of this language is to help students understand compiler construction concepts by working with a simplified programming language.

Style of Syntax

The style of the language is block-structured and statemend based, similar to C++.

It supports variable declarations, assignments, conditional statements, loops, and print statements using paranthesis and codo blocks.

Reason for choosing Keywords

The keywords are inspired by the "Bankai" theme to make the language unique and interesting. Although the names are customized, the functionality of the keywords remains similar to standard C++ keywords, making the language easy to understand.

Keywords

1- int_bankai
2- float_bankai
3- if_bankai
4- else_bankai
5- while_bankai

Operators

1- add+ (addition)
2- sub- (subtraction)
3- == (equality comparison)

Punctuation

1- ; (statement terminator)
2- {b (start of block code)

2) Grammer Definitions

Non-Terminals Used

Program
StmtList
Stmt
Decl
Assign
IfStmt
LoopStmt
IOStmt
Expr
Term
Factor
[redacted]

Terminals

All keywords, operators, punctuations (in my language) and others like (id, num string) etc-

Grammer Rules

1- Program Structure

Program —> Stmt List

2- Statement List

StmtList —> Stmt StmtList
StmtList —> ε

3- General Statement

Stmt —> Decl
Stmt —> Assign
Stmt —> IfStmt
Stmt —> LoopStmt
Stmt —> IOStmt

4- Variable Declaration

Decl → int_bankai id ;

Decl → float_bankai id ;

5- Assignment statement

Assign → id = Expr ;

6- Conditional Statement

IfStmt → if_bankai (Expr == Expr) { b StmtList }

IfStmt → if_bankai (Expr == Expr) {b StmtList} else_bankai {b StmtList }

7- Loop Statement

LoopStmt → while_banfai (Expr == Expr) {b StmtList }

LoopStmt → for_bankai (Assign Expr == Expr ; Assign) {b StmtList }

8- Input / Output Statement

IOStmt → cin_bankai >> id;

IOStmt → display-bankai << Expr ;

9- Expressions

Expr → Expr addt Term

Expr → Expr sub- Term

Expr → Term

0 - Terms

Term → Term mul* Factor

Term → Term div/ Factor

Term → Term remi. Factor

Term → Factor

11- Factor

Factor → id

Factor → num

3) Production Rules

Program -> Stmt List

StmtList -> Stmt StmtList

Stmt List -> E

Stmt -> Decl

Stmt -> Assign

Stmt -> If Stmt

Stmt -> LoopStmt

Stmt -> IO Stmt

Decl -> int_bankai id;

Decl -> float_bankai id;

Assign -> id = Expr;

If Stmt -> if_bankai (Expr == Expr) {b StmtList }

If Stmt -> if_bankai (Expr == Expr){b StmtList} else_bankai [redacted] {b StmtList

Loopstmt -> while_bankai (Expr == Expr) {b StmtList}

LoopStmt -> for_bankai (Assign Expr==Expr; Assign) {b StmtList}

Io Stmt -> cin_bankai >> id;

Io Stmt -> display_bankai << Expr;

Expr -> Expr add+ Term | Expr sub- Term | Term

Term -> Term mul* Factor | Term div/ Factor | Term rem% Factor | Factor

Factor -> id | num

4) Follow and First

Non-Terminal	First	Follow
Program	{int_bankai, float_bankai, if_bankai, while_bankai, for_bankai, cin_bankai, display_bankai, id}	{$}
IOStmt	{cin_bankai, display_bankai}	{int_bankai, float_bankai, if_bankai, while_bankai, for_bankai, cin_bankai, display_bankai, $}

5) Ambiguity Check

There is the "Dangling Else" problem. The ambiguity occurs in conditional statements with optional else.

Grammer Rules Involved

IfStmt -> if_bankai (Expr == Expr){b StmtList}
IfStmt -> if_bankai (Expr == Expr){b StmtList} else_bankai {b StmtList}

Example

```
if_bankai (a == b) {b
        if_bankai (c == d) {b
                display_bankai << a;

        }

} else_bankai {b
        display_bankai << b;

}
```

Ambiguity

It is unclear which if_bankai the else_bankai belongs to

1- The inner if_bankai
2- The outer if_bankai

Both interpretations are valid according to the grammer, which makes it ambiguous.

How to Resolve this Ambiguity

This ambiguity is resolved by

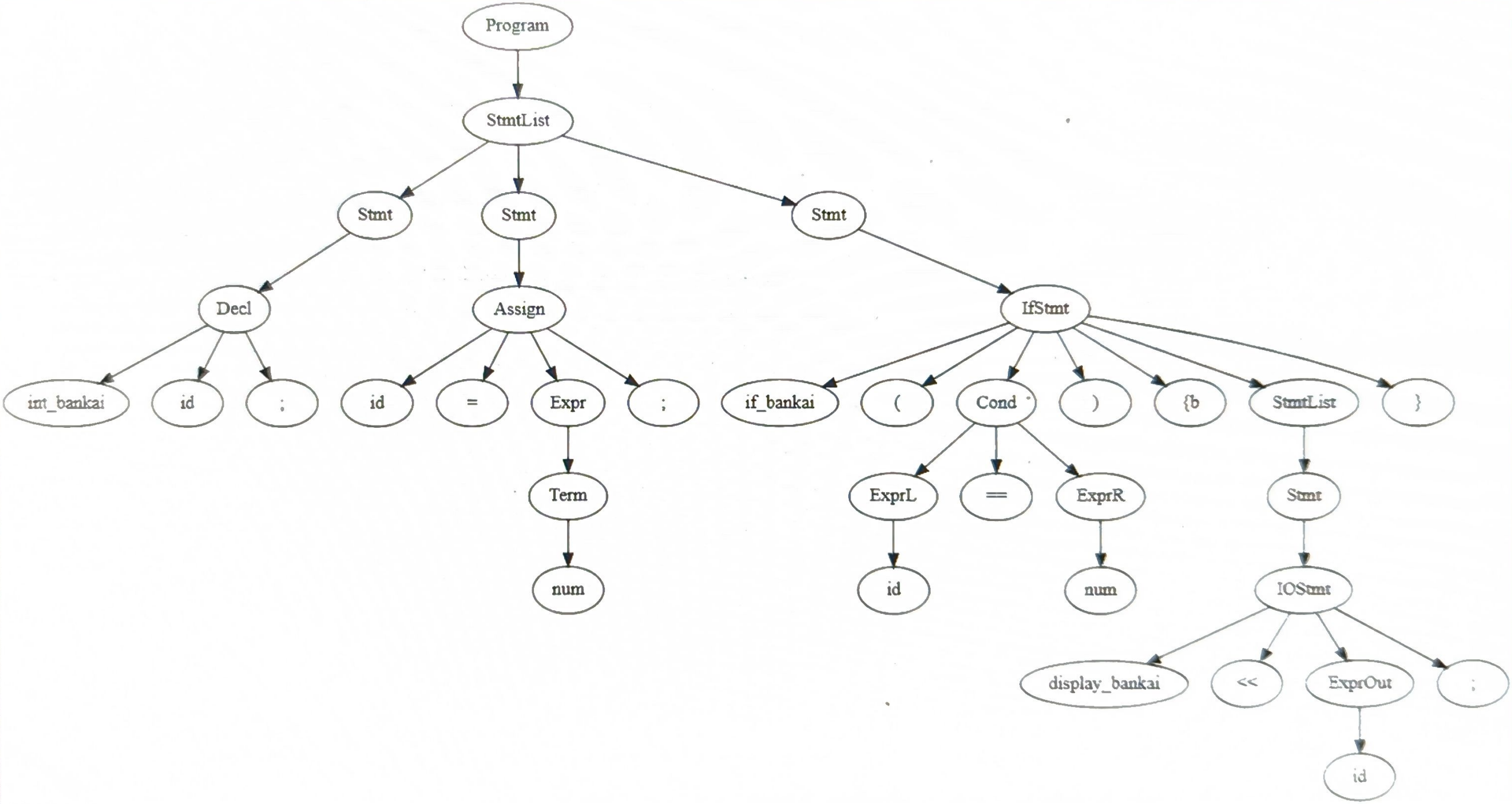
1- Associating the else_bankai with the nearest unmatched if_bankai
2- Using parser rules that always bind else to the closest if

6) Parse Tree Construction

Valid Program Fragment

```
int_bankai a;
  a=5;
if_bankai (a==5) {b
        display_bankai<<a;
   }
```

Parse Tree


Program
StmtList
Stmt
Stmt
Stmt
Decl
Assign
IfStmt
int_bankai
id
;
id
=
Expr
;
if_bankai
(
Cond
)
{b
StmtList
}
Term
num
ExprL
==
ExprR
Stmt
id
num
IOStmt
display_bankai
<<
ExprOut
;
id